

GitHub Actionsの使い方を噛み砕いて理解する

「pushしたら自動でテストが走る」「mainにマージしたら自動でデプロイされる」といった仕組みを実現するのがGitHub Actionsです。用語が多く感じますが、構造はシンプルです。

1. まず全体像をアナロジーで理解する 🏭

GitHub Actionsは「**工場の自動生産ライン**」に例えるとわかりやすいです。

工場の用語	GitHub Actionsでの対応
「材料が届いたら生産開始」という起動条件	on （トリガー：いつ実行するか）
1つの生産ライン全体	workflow （ <code>.github/workflows/</code> 配下の1つの <code>.yaml</code> ファイル）
ラインの中の「工程」（検品工程、梱包工程など）	job （並行 or 順番に実行される作業のまとまり）
1つの工程の中の「作業手順」	step （実際にコマンドを実行する1つ1つの手順）
既製の作業道具（自分で作らず借りてくる）	action （ <code>actions/checkout</code> のような再利用可能な部品）
実際に作業する「作業員・作業スペース」	runner （GitHubが用意する実行環境の仮想マシン）

➡ **workflow(工場全体) > job(工程) > step(作業手順)** という3階層構造だけ覚えれば、YAMLの構造がすぐ理解できます。

2. シンプルなworkflowを実際に見てみる

以下は「`main` ブランチにpushされたら、テストを自動実行する」という最もシンプルな例です。

```
# .github/workflows/test.yml

name: Run Tests # ①workflowの名前(GitHubの画面に表示される)

on: # ②いつ実行するか(トリガー)
  push:
    branches: [main]

jobs: # ③jobの定義開始
  test: # jobの名前(自由に命名可)
    runs-on: ubuntu-latest # ④どんな環境(runner)で実行するか

    steps: # ⑤具体的な作業手順(上から順に実行)
      - name: リポジトリのコードを取得
        uses: actions/checkout@v4

      - name: Node.jsをセットアップ
        uses: actions/setup-node@v4
        with:
          node-version: "20"

      - name: 依存パッケージをインストール
        run: npm install

      - name: テストを実行
```

```
run: npm test
```

3. 1行ずつ噛み砕いて解説

① `name` : workflowの名前

```
name: Run Tests
```

GitHubの「Actions」タブに表示される名前です。省略も可能ですが、複数workflowを管理する時のためにつけておくのが基本です。

② `on` : いつ実行するか（トリガー）

```
on:
  push:
    branches: [main]
```

「`main` ブランチにpushされた時」に発火する、という意味です。他にもよく使うトリガーがあります。

トリガー	意味
<code>push</code>	指定ブランチへのpush時
<code>pull_request</code>	PR作成・更新時
<code>workflow_dispatch</code>	手動で「実行」ボタンを押した時
<code>schedule</code>	決まった時間に定期実行（cron形式）

複数のトリガーを組み合わせる例

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch: {}
```

③ `jobs` : 仕事のまとめ

```
jobs:
  test:
    ...
```

`test` という名前の`job`を1つ定義しています。jobは複数定義することもでき、デフォルトでは**並行実行**されます（依存関係を指定すれば順番も制御可能）。

④ `runs-on` : どの実行環境を使うか

```
runs-on: ubuntu-latest
```

GitHubが用意している仮想マシン(runner)を指定します。`ubuntu-latest` が最も一般的で、他に `windows-latest` や `macos-latest` もあります。

⑤ `steps` : 具体的な作業内容

```
steps:
  - name: リポジトリのコードを取得
    uses: actions/checkout@v4
```

`steps`内の各項目には主に2つの書き方があります。

キーワード	意味	例
<code>uses</code>	誰かが作った既製のaction(部品)を使う	<code>actions/checkout@v4</code>
<code>run</code>	自分でコマンドを直接実行する	<code>run: npm install</code>

`actions/checkout` は「このリポジトリのコードをrunner上に持ってくる」という、ほぼ全workflowで最初を書く定番actionです。

4. jobを複数に分ける例（テスト → デプロイ）

実務では「テストが通ったらデプロイする」のように、jobを分けて**依存関係**を持たせることがよくあります。

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test

  deploy:
    needs: test                # ← testジョブが成功した後にのみ実行される
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "デプロイ処理をここに書く"
```

`needs: test` が「工程の順番」を表しています。これが無いと `test` と `deploy` は同時に走ってしまうので注意が必要です。

5. Secrets(秘密情報)の使い方

APIキーやパスワードなど、YAMLに直接書けない情報はGitHubリポジトリの **Settings** → **Secrets** に登録し、以下のように参照します。

```
steps:
  - name: AWSにログイン
    env:
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
      AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
    run: aws s3 ls
```

`{{ secrets.変数名 }}` という書き方がGitHub Actions特有の記法(式構文)です。ログにも自動でマスクされるため、誤って値が漏れる心配がありません。

6. これまで扱ってきたECS/OIDC構成との関係

`snatch_system` のようなAWS ECSへのデプロイ構成では、上記の基本構成に加えて以下の要素が追加されているだけで、**土台の考え方(on → jobs → steps)**は同じです。

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write    # OIDC認証に必要な権限
      contents: read
    steps:
      - uses: actions/checkout@v4

      - name: AWS認証(OIDC、パスワード不要)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
          aws-region: ap-northeast-1

      - name: ECRにログインしてイメージをpush
        run: |
          aws ecr get-login-password | docker login --username AWS --password-stdin <ECRのURL>
          docker build -t myapp .
          docker push <ECRのURL>/myapp

      - name: ECSサービスを更新
        run: aws ecs update-service --cluster my-cluster --service my-service --force-new-deployment
```

`permissions: id-token: write` と `aws-actions/configure-aws-credentials` が「**OIDCでAWSの一時的な認証情報を安全に取得する**」ための追加要素です。長期のアクセスキーをSecretsに保存せずに済むため、より安全な構成として採用されています。

7. つまづきやすいポイント

症状	原因・対処
workflowが全く実行されない	<code>.github/workflows/</code> 配下に置かれているか、YAMLのインデントが崩れていないか確認
<code>npm test</code> が「コマンドが見つからない」で失敗	<code>actions/setup-node</code> でNode.jsをセットアップし忘れている
jobが想定と違う順番で走る	<code>needs</code> を指定し忘れている(デフォルトは並行実行)
Secretsが空になる	Secretsの登録場所(リポジトリ/Organization)の階層違い、または名前のtypo
PRからのworkflowでSecretsが使えない	フォークからのPRではセキュリティ上Secretsが渡されない仕様(意図的な制限)

8. まとめ

- GitHub Actionsの構造は **workflow(全体)** > **job(工程)** > **step(作業手順)** の3階層
- `on` で「いつ動くか」、`runs-on` で「どこで動くか」、`uses/run` で「何をするか」を指定する
- jobを分けて `needs` で繋げることで、「テストが通ったらデプロイ」のような順序制御ができる
- 秘密情報は `${{ secrets.変数名 }}` で安全に参照する
- 実際のECS/OIDCデプロイも、この基本構造に「AWS認証」「Docker」「ECS更新」のstepを追加しているだけ